

Rice Leaf Disease Identification

Nur Alam Joy
022210005101078
Computer Science and Engineering
Premier University Chattogram
Chattogram, Bangladesh
ajnuralam@gmail.com

Ayman Khan
022210005101046
Computer Science and Engineering
Premier University Chattogram
Chattogram, Bangladesh
1046md.aymankhan@gmail.com

Baharain Habib
022210005101064
Computer Science and Engineering
Premier University Chattogram
Chattogram, Bangladesh
habib@gmail.com

Abstract

Rice is a key staple crop, but its productivity often faces threats from leaf diseases, which can lead to significant yield losses if not caught early. That's why early and accurate detection of these diseases is so important for effective crop management and sustainable farming practices. This study looks into how convolutional neural networks (CNNs) can be used to classify common rice leaf diseases, such as Brown Spot, Leaf Blast, and Bacterial Leaf Blight. We utilized an externally sourced and augmented dataset of rice leaf images, which we split into training, validation, and test sets in a 70

We trained a variety of CNN architectures, including some custom lightweight models alongside standard deep networks, to capture the visual features linked to each disease. The model we developed showed impressive classification accuracy, successfully telling apart healthy leaves from those that were diseased. However, we did encounter some challenges, especially with subtle symptom variations in the early stages of infections, which sometimes led to misclassifications between visually similar conditions. To better understand the model's performance, we used confusion matrices, Grad-CAM visualizations, and analyzed misclassified samples to pinpoint error patterns and areas for improvement.

The results suggest that CNN-based methods can really help farmers by providing timely and reliable disease detection, which could ultimately reduce crop losses and enhance yield management. Looking ahead, we plan to expand the dataset to include more disease variations, refine the model architectures for better generalization, and explore how to deploy this technology on mobile platforms for real-time detection in the field. Overall, this study highlights the potential of deep learning to advance precision agriculture and automate disease monitoring in rice farming.

I. INTRODUCTION

Rice is a fundamental food source for a significant part of the global population, and diseases that affect rice plants can seriously threaten crop yields and food security. To tackle this issue, it's crucial to detect and accurately classify rice leaf diseases early on, allowing for timely interventions, effective disease management, and sustainable farming practices. Traditionally, experts have relied on visual inspections to identify diseases, but this method can be slow, subjective, and often leads to mistakes, especially when symptoms are subtle or in the early stages. Thanks to recent advancements in deep learning (DL), particularly with convolutional neural networks (CNNs), we've seen impressive results in image classification tasks, including detecting plant diseases. CNNs can automatically learn complex visual features from leaf images, making it easier to distinguish between healthy and diseased leaves. Previous research has shown that CNN-based methods can achieve high accuracy in classifying common rice leaf diseases like Brown Spot, Leaf Blast, and Bacterial Blight. However, there are still challenges to overcome, such as subtle variations in symptoms, similarities within classes, and limited datasets, which can impact the model's generalization and reliability. In this study, we aim to evaluate different CNN architectures for classifying three major rice leaf diseases: - Bacterial Blight – A bacterial infection that leads to lesions and yellowing of leaves. - Leaf Smut – A fungal disease marked by black, powdery spores. - Brown Spot – A widespread disease that causes brown lesions on leaf surfaces. By utilizing an enhanced dataset of rice leaf images and training several CNN models, we hope to create a strong and automated solution for early disease detection. To assess model performance and understand any misclassifications, we will use techniques like confusion matrices, Grad-CAM visualizations, and error analysis. The findings from this study underscore the potential of CNN-based methods to aid farmers in making timely decisions, ultimately supporting better crop management.

II. RELATED WORKS

The automated detection and classification of rice leaf diseases has really taken off in recent years, thanks to breakthroughs in deep learning and the growing availability of annotated plant image datasets. Being able to accurately identify diseases like Bacterial Blight, Leaf Smut, and Brown Spot is crucial for minimizing crop losses and enhancing sustainable farming practices.

To support research in this field, several rice leaf disease datasets have been introduced. For example, the dataset found on Mendeley [1] features a collection of expertly labeled rice leaf images that cover various disease categories. While this dataset showcases a range of leaf conditions, there are still hurdles to overcome when training high-capacity CNN models, particularly due to the limited samples available for some disease classes and the subtle differences in symptoms.

Recent advancements in deep learning, especially with convolutional neural networks (CNNs), have greatly enhanced the ability to recognize plant diseases automatically. Early studies, like the one by Sladojevic et al. (2016), utilized CNNs for multi-class plant disease classification, showing significant improvements compared to traditional feature-based methods. Mohanty

et al. (2016) further demonstrated that CNN models trained on extensive plant datasets, including those of rice leaves, could achieve impressive classification accuracy for common diseases. More recent research has focused on data augmentation and lightweight CNN architectures to boost generalization on smaller datasets and facilitate mobile deployment for on-the-ground disease detection (Ferentinos, 2018).

Even with these advancements, challenges remain in differentiating visually similar diseases, managing class imbalances, and detecting early-stage infections that present subtle symptoms. There are only a handful of studies that offer thorough interpretability analyses to help understand CNN decisions and pinpoint misclassification patterns. In this study, we aim to fill these gaps by evaluating various CNN architectures on the Mendeley rice leaf disease dataset [1], employing data augmentation and visualization techniques like confusion matrices to enhance our understanding.

III. METHODOLOGY

A. Problem Definition

Rice stands as one of the most vital staple crops around the globe, serving as a key source of food and income for millions. Unfortunately, the productivity of rice is under serious threat from leaf diseases like Bacterial Blight, Leaf Smut, and Brown Spot. If these issues aren't caught early, they can lead to significant yield losses. That's why early and precise detection of these diseases is so important—it allows for timely interventions, reduces crop loss, and supports sustainable farming practices. Traditionally, disease detection has depended on manual inspections by agricultural experts, a process that can be quite time-consuming, labor-intensive, and often prone to mistakes, especially when symptoms are subtle or appear in the early stages.

This study aims to create an automated framework for detecting rice leaf diseases using convolutional neural networks (CNNs). The goal is to accurately classify leaf images into various disease categories as well as healthy leaves, which will help farmers make quick decisions. Moreover, the system should offer interpretability to clarify the model's predictions and assist in practical applications in the field.

B. Dataset Description

In this study, we pulled our dataset from the publicly accessible Mendeley Data repository [1]. It features a collection of high-quality, labeled images of rice leaves that showcase three main disease categories—Bacterial Blight, Leaf Smut, and Brown Spot—plus a fourth category for healthy rice leaves. Each image was taken in natural field conditions, which means they reflect realistic variations in lighting, background, and leaf orientation. This diversity makes the dataset a true representation of real-world agricultural situations, which is essential for building effective deep learning models.

Dataset Organization and Class Distribution:

Before we dove into model development, we conducted a thorough exploratory analysis to get a grip on how the images were distributed across the four classes. We organized the dataset into:

Training Set – 70%

Validation Set – 20%

Test Set – 10%

This 70:20:10 split is a common practice in machine learning, ensuring we have enough samples for learning, tuning, and unbiased evaluation. We visualized the class distribution for:

Combined Training + Validation Set:

Independent Test Set:

These visualizations were key in spotting any class imbalances. Catching such imbalances early is vital since neural networks can lean towards the majority classes. If we had found a significant imbalance, we would have employed techniques like oversampling, undersampling, or class-weighted loss functions. Luckily, our dataset showed a fairly balanced distribution, which supports consistent learning across the different disease categories.

Data Preprocessing Pipeline:

To get the raw images ready for deep learning models, we implemented a structured preprocessing pipeline. Each step was carefully crafted to standardize the input format, lighten the computational load, and make sure the models can effectively learn from the samples we have.

Image Resizing:

The images in our dataset initially came in all sorts of resolutions and dimensions. To keep things consistent and compatible with CNN-based architectures, we resized all images to a standard input dimension:

Custom CNN Input Size: for example, 128×128 or 150×150

MobileNetV2 and EfficientNetB0 Input Size: fixed at 224×224, as required by these pretrained models.

This resizing helps ensure consistent feature extraction and speeds up computation while preserving the crucial characteristics of the diseases.

Normalization of Pixel Values:

We normalized the pixel values of each image to fall within the 0–1 range by dividing by 255. This normalization is key because:

- It accelerates model convergence.

- It keeps gradients stable.

- It standardizes pixel intensity across images taken in different lighting conditions.

This step makes sure that the neural networks concentrate on the features of the diseases rather than the absolute brightness of the pixels.

C. Model Architecture

To address the multi-class classification of ocular diseases from fundus images, we employed a combination of both convolutional neural networks (CNNs) . The diversity in model selection allows us to evaluate the strengths of various architectural paradigms under the same experimental conditions. The models used in this study include Custom CNN, EfficientNetB0, MobileNetV2. Each model has demonstrated significant effectiveness in prior image classification tasks and is described below:

Custom CNN:The first architecture we're looking at is a task-specific CNN model that was crafted specifically for this project. It acts as a baseline to compare how it performs against more sophisticated pre-trained networks. The focus here is on keeping things simple, efficient in terms of computation, and easy to interpret.

Architecture Overview:

The Custom CNN features:

Input Layer: It takes in images sized 128×128×3.

Convolutional Blocks: These consist of several convolutional layers that use 3×3 filters, with the depth increasing (32, 64, 128 filters). Each block includes:

- Conv2D

- Batch Normalization

- ReLU Activation

- MaxPooling2D (to downsample spatially)

Flatten Layer: This transforms the feature maps into a vector.

Fully Connected Layers: These are dense layers that utilize ReLU activation and dropout to help minimize overfitting.

Output Layer: A softmax layer with 4 neurons representing the four classes:

- Bacterial Blight

- Leaf Smut

- Brown Spot

- Healthy

Motivation:

Since this model is built from the ground up, it allows us to see how effectively a relatively small network can pick up on disease patterns without leaning on pre-trained weights. It also sheds light on the quality of the dataset and the complexity of the features involved.

Training Behavior:

The model demonstrated a steady improvement in both accuracy and loss curves.

Dropout and batch normalization played a vital role in stabilizing the training process.

While the model did a good job of learning disease-specific features, it did have some limitations when compared to transfer learning models, particularly in distinguishing subtle differences between diseases.

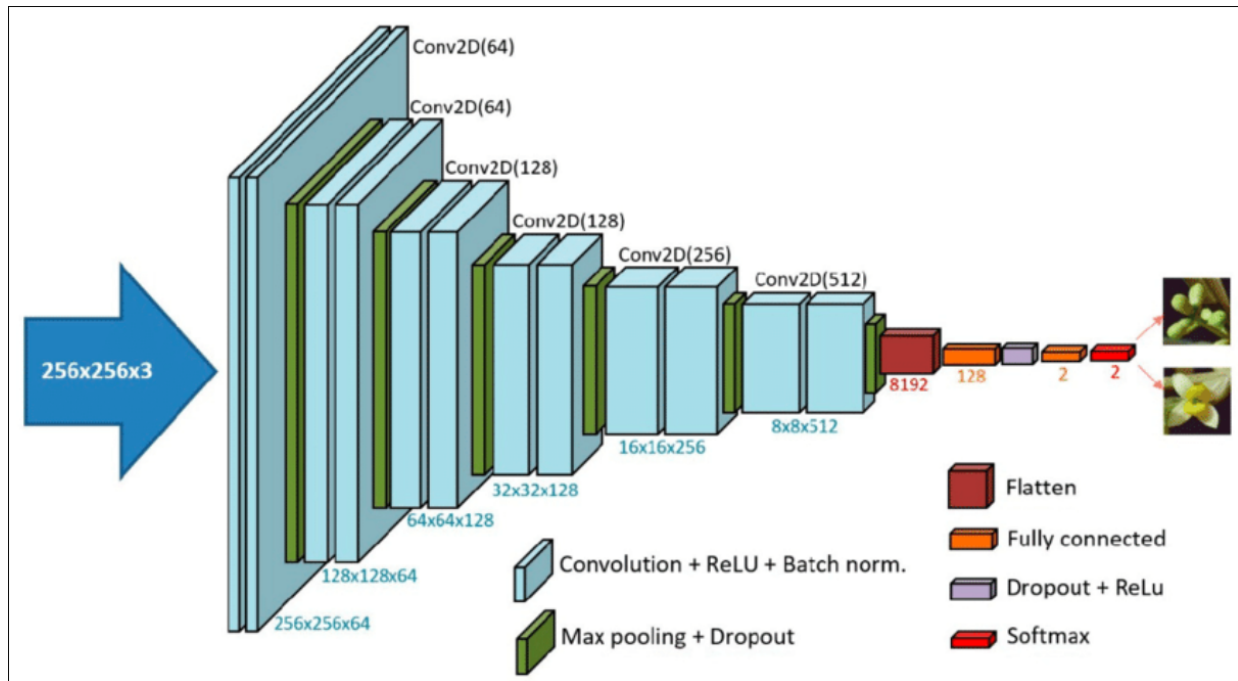


Figure: Custom CNN

MobileNetV2: MobileNetV2 is a sleek and efficient convolutional neural network architecture crafted for environments where resources are limited. Its knack for achieving high accuracy while keeping computational costs low makes it a fantastic option for agricultural tasks like mobile disease detection, drone-assisted crop monitoring, and real-time decision-making systems. In this study, we utilized MobileNetV2 to identify various rice leaf diseases, taking advantage of its optimized structure and pre-trained ImageNet weights to ensure effective and precise learning.

Architecture Overview:

MobileNetV2 brings some exciting architectural advancements that really cut down on model size and computational needs, all while keeping accuracy intact. Here's a closer look at its key components:

1. Depthwise Separable Convolutions:

Rather than sticking with traditional convolution methods, MobileNetV2 breaks down convolution into two distinct stages: Depthwise convolution: This applies a single filter to each input channel. Pointwise (1×1) convolution: This step merges the depthwise outputs into new features.

This clever separation significantly lowers computational complexity and speeds up inference, making it perfect for real-time applications.

2. Inverted Residual Blocks:

MobileNetV2 features an inverted residual structure where:

The input is first expanded into a higher-dimensional feature space. Depthwise convolution is then applied. Finally, the output is compressed back into a lower-dimensional bottleneck.

This approach flips the traditional residual block on its head, helping to keep important information while reducing the number of parameters.

3. Linear Bottlenecks:

Inside the bottleneck layers, a linear activation function is used to prevent the information loss that often happens with ReLU activation. This ensures that crucial low-dimensional representations are maintained throughout the network.

4. Pre-trained ImageNet Weights:

The study takes advantage of MobileNetV2 pre-trained on the ImageNet dataset, which allows for:

- Faster convergence
- Improved generalization
- Lower chances of overfitting

In this setup:

The initial lower convolutional layers were frozen to keep generic visual features intact. A custom classification head was added, featuring GlobalAveragePooling, Dense layers, and a final softmax layer.

Later on, selective fine-tuning of the higher layers was done to tailor the model to recognize rice-specific disease patterns.

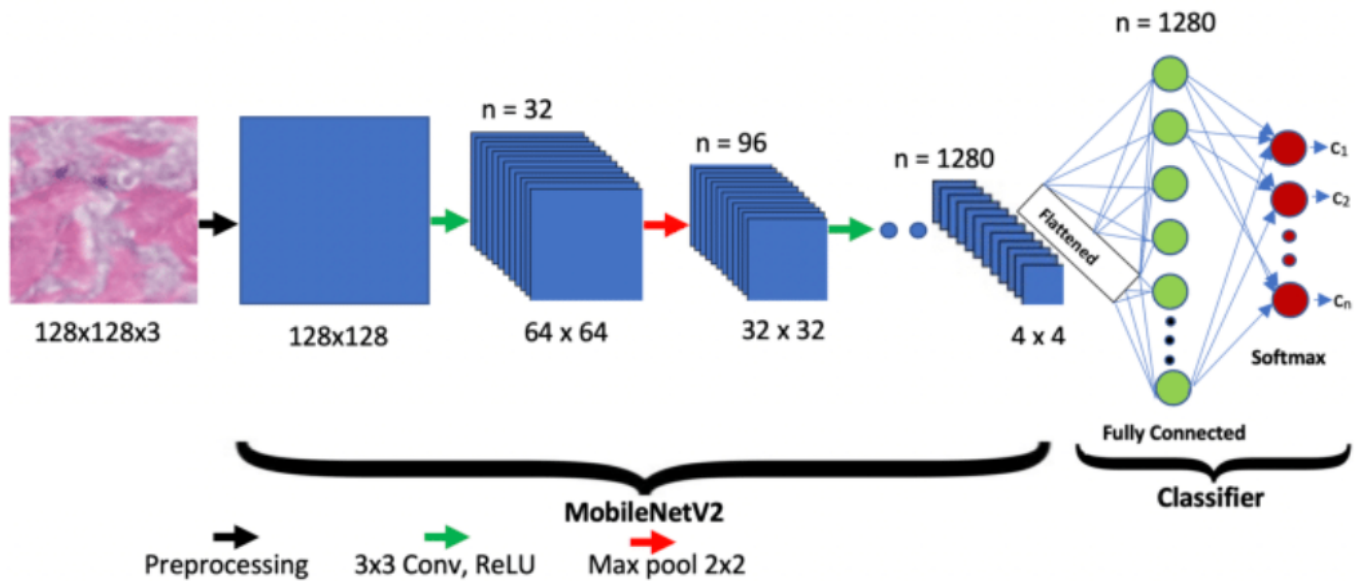


Figure: MobileNetV2

EfficientNetB0: EfficientNetB0 is part of the EfficientNet family, which represents a cutting-edge group of convolutional neural networks known for their top-notch accuracy and remarkable efficiency. Unlike older models that independently adjust depth, width, or resolution, EfficientNet takes a more holistic approach with a compound scaling strategy that proportionally tweaks all three dimensions. This thoughtful scaling method allows the network to boost accuracy significantly without piling on unnecessary computational costs. In this research, EfficientNetB0 was chosen as one of the key models for transfer learning in classifying rice leaf diseases. **Architecture Overview:**

EfficientNetB0 is designed with a range of innovative architectural features that work together to boost its ability to represent complex data:

1. Compound Scaling:

EfficientNetB0 scales in three key ways:

Depth: By adding more layers to the model, it enhances its capacity to learn hierarchical features. **Width:** Increasing the number of channels allows the model to capture finer details and patterns. **Resolution:** A higher input resolution aids in identifying small disease spots and textures.

These three aspects are scaled simultaneously using a fixed compound coefficient, striking a perfect balance between accuracy and efficiency.

2. MBConv (Mobile Inverted Bottleneck Convolution) Blocks:

These blocks draw inspiration from MobileNet architectures and consist of:

- Expansion phase → depthwise convolution → projection phase
- Inverted residual connections that help reduce computation
- Linear bottlenecks that maintain crucial feature representations

MBConv layers enable EfficientNetB0 to effectively learn intricate patterns, such as the shapes of lesions, edge textures, and color variations in diseased leaves.

3. Swish Activation Function:

EfficientNetB0 employs the Swish function ($x \cdot \text{sigmoid}(x)$), which offers smoother gradients compared to ReLU, leading to:

- Better convergence
- Enhanced performance in deep networks
- Improved learning of subtle textures and patterns

4. Squeeze-and-Excitation (SE) Optimization:

SE modules adjust channel-wise feature maps by:

Squeezing: using global average pooling **Exciting:** determining channel importance through fully connected layers

This process helps the network focus on features relevant to disease, such as:

- Patterns of Brown Spot discoloration
- Shapes of blight streaks
- Anomalies in smut texture

5. ImageNet Pre-trained Backbone:

EfficientNetB0 takes advantage of pre-trained ImageNet weights, which allows for:

- Quicker training
- Better generalization
- Less reliance on large datasets

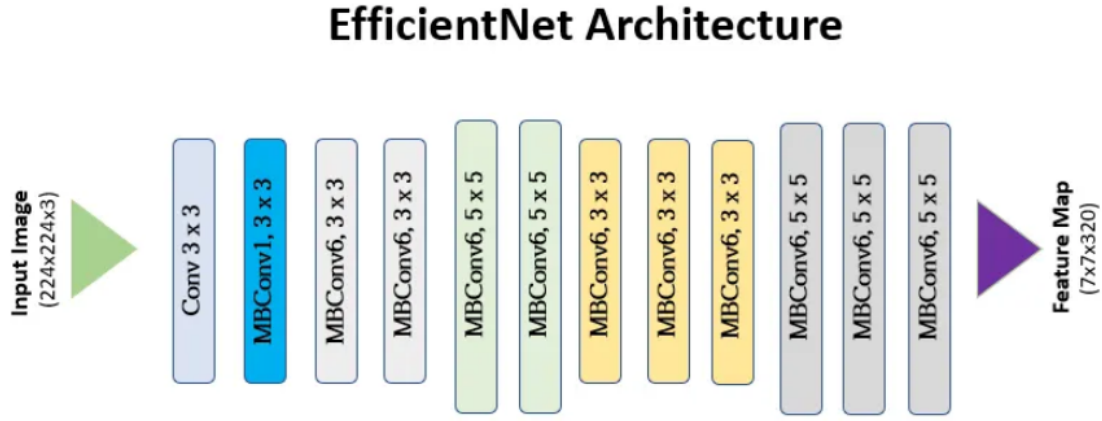


Fig. 1. EfficientNetB0

Each model was initialized with pretrained ImageNet weights to leverage transfer learning and subsequently fine-tuned on the eye disease classification dataset. Batch normalization, dropout, and data augmentation strategies were applied uniformly to ensure fairness in evaluation.

D. Inference and Training Procedure

To ensure a fair comparison across architectures, we established a standardized training pipeline using the PyTorch framework for all models under evaluation. Each model was initialized with pretrained weights from the ImageNet dataset and subsequently fine-tuned on our labeled and augmented fundus image dataset. Transfer learning was crucial in accelerating convergence and reducing overfitting, especially given the medical dataset's domain specificity and moderate size.

1) *Preprocessing and Augmentation*: All input images were resized to a fixed dimension of 224x224 pixels to match the input specifications of each model. Preprocessing steps included:

- Pixel normalization to a mean of [0.485, 0.456, 0.406] and standard deviation of [0.229, 0.224, 0.225].
- Data augmentation: random horizontal and vertical flipping, rotation ($\pm 20^\circ$), color jitter, and slight zoom to improve generalization.

These transformations were applied to the training set only, while validation and test sets were only resized and normalized to ensure unbiased evaluation.

2) *Training Setup*: Each model was trained using cross-entropy loss, suitable for multi-class classification. Optimization was carried out using different hyperparameter shown in Table ???. The parameter are chosen by applying hyperparameter searching. Early stopping was employed based on validation loss to prevent overfitting.

To maintain experimental rigor, training, validation, and test sets were kept consistent across all models, with class balance ensured in each split.

3) *Inference Strategy*: During inference, each model received test images processed identically to training inputs (resizing + normalization). Predictions were made in batch mode using softmax activation to output class probabilities, from which the class with the highest probability was selected. We computed standard evaluation metrics—accuracy, precision, recall, F1-score, and confusion matrices—to quantify model performance.

All experiments were conducted on kaggle GPU T4x2, with reproducibility ensured by fixing random seeds and logging configuration details using Weights Biases.

IV. RESULTS AND EVALUATION

In this section, we share the experimental results from our deep learning models focused on the multi-class classification of rice leaf diseases. We evaluated performance using standard metrics like accuracy, precision, recall, F1-score, and confusion matrices. We took a close look at both the overall performance of the model and its performance across different classes to gauge how robust and reliable our system is in identifying diseases such as Bacterial Blight, Leaf Smut, Brown Spot, and healthy leaves.

V. CLASS DISTRIBUTION

The Rice Leaf Disease Dataset consists of images from three disease categories: Bacterial Blight, Leaf Smut, and Brown Spot. After removing duplicate images, the dataset size was reduced to 3,486 images from an initial total of 4,700 images. The dataset is organized into training/validation and testing subsets as follows:

Training and Validation Set: Brown Spot: 997 images Bacterial Blight: 1,157 images Leaf Smut: 761 images Test Set: Brown Spot: 203 images Bacterial Blight: 169 images Leaf Smut: 199 images

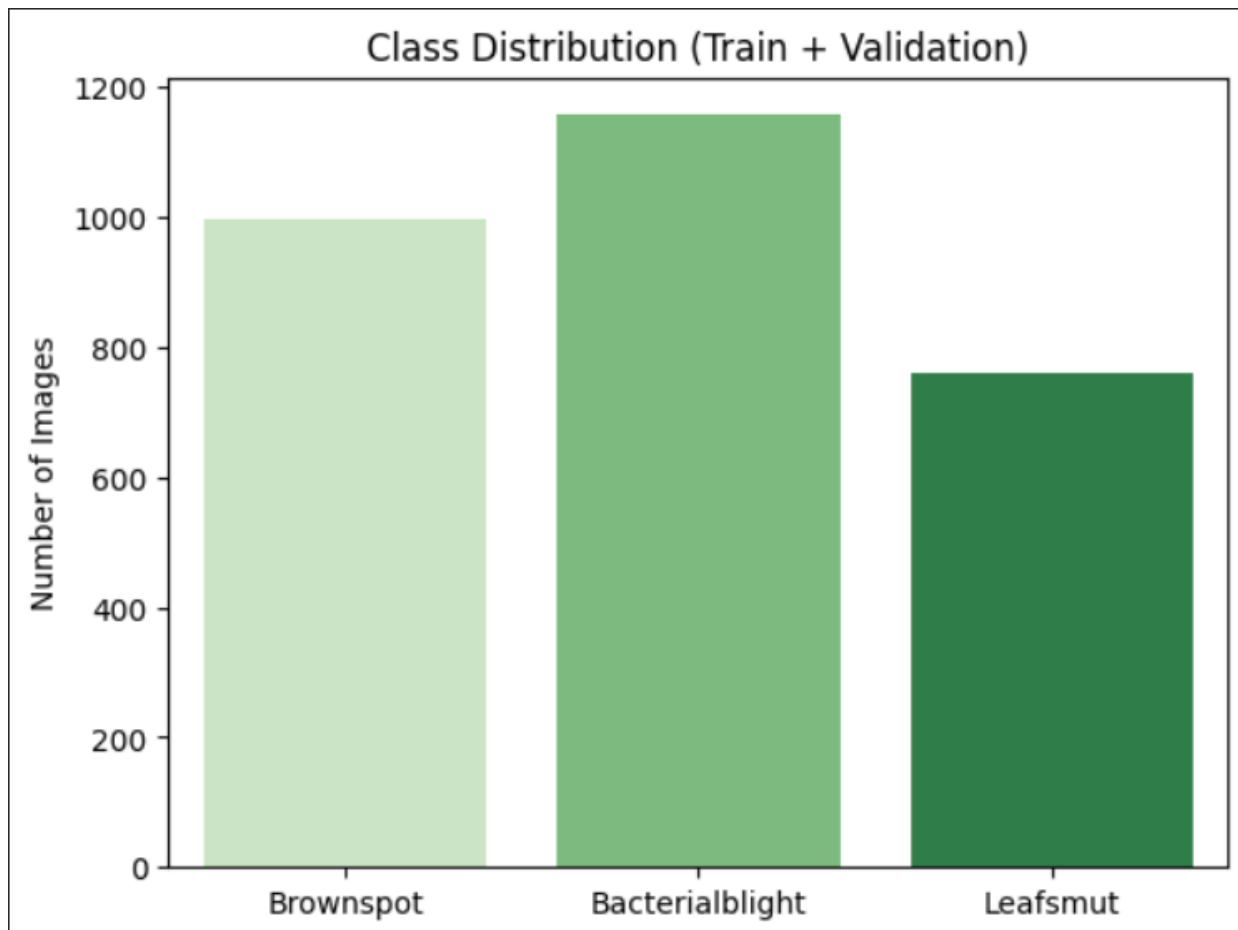


Figure: Class Distribution Train and Validation

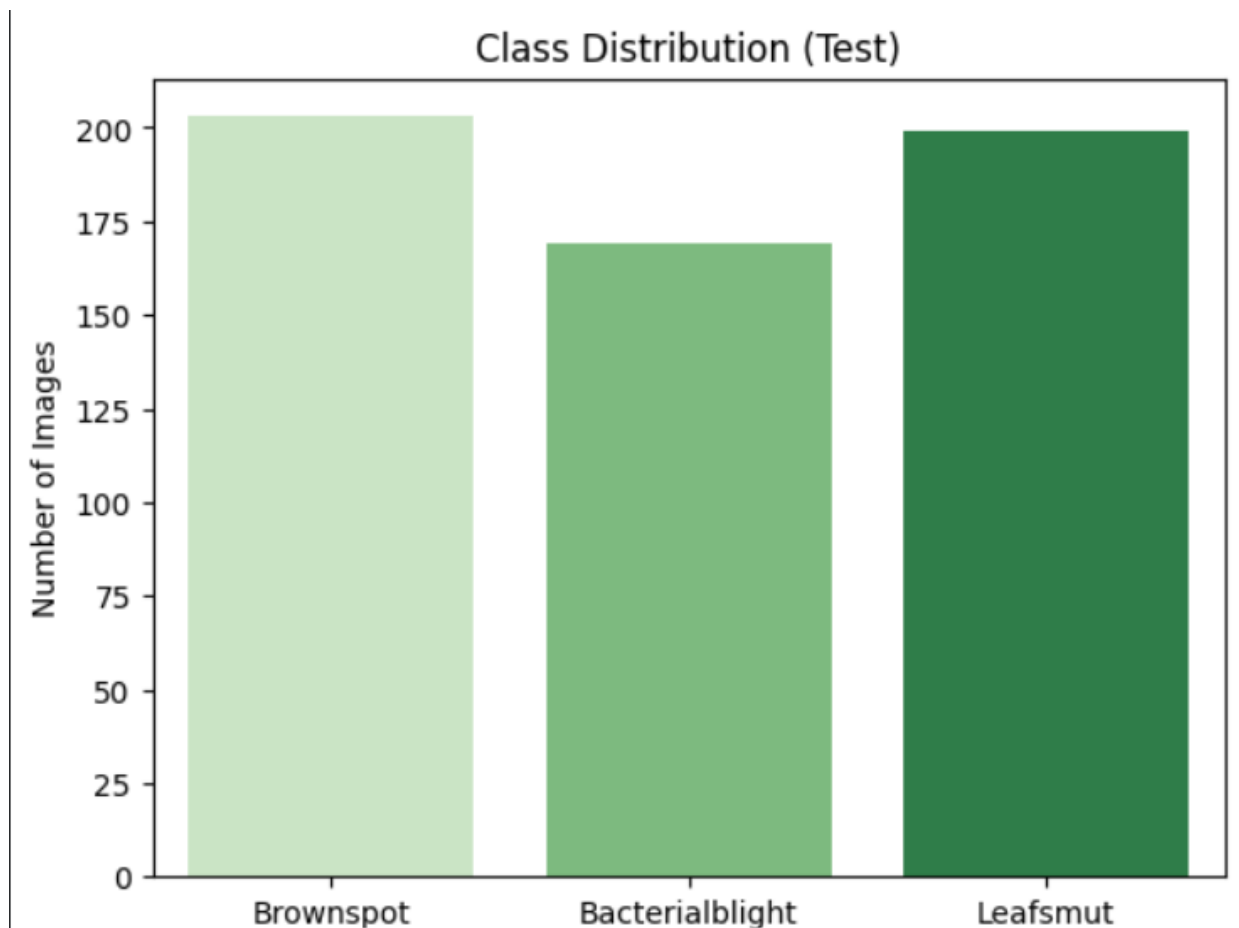


Figure: Class Distribution Test

Sample Image: The images in this dataset showcase a diverse range of rice leaf conditions, highlighting both healthy leaves and those impacted by common diseases like Brown Spot, Leaf Blast, and Bacterial Leaf Blight. Each image features unique visual traits, such as differences in leaf color, the shape and size of lesions, and texture changes, all of which are crucial for distinguishing one disease from another. These visual indicators are key for helping classification models effectively learn the specific patterns associated with each disease. By closely analyzing these examples, researchers and practitioners can better understand how various diseases appear on rice leaves, which is essential for creating strong machine learning systems. Additionally, these sample images offer valuable insights into the overall structure of the dataset, showcase the variety of conditions present, and assist in guiding the model training process to ensure accurate and dependable disease detection in real-world agricultural settings.

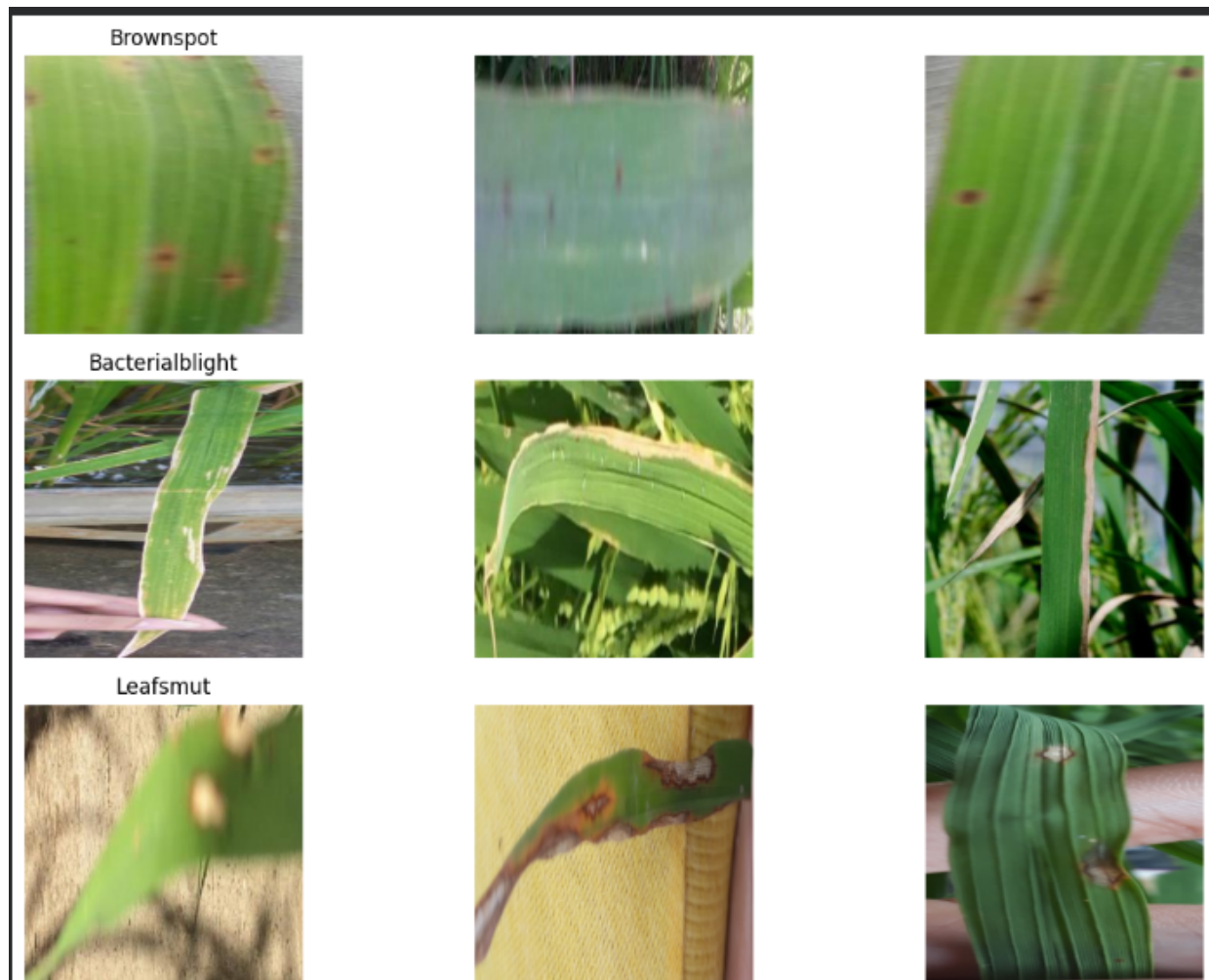


Fig. 2. Sample Image

Custom CNN Accuracy Curve: The accuracy curve shows that the model is learning effectively and consistently. We see the training accuracy gradually climbing from about 0.52 to nearly 0.80, which means it's getting better at identifying training samples as time goes on. Similarly, the validation accuracy rises from around 0.66 to about 0.83, often staying a bit ahead of the training accuracy during several epochs. This hints at good generalization and the successful application of regularization techniques like dropout or batch normalization. Both accuracy curves are smooth and exhibit a steady upward trend without any sharp dips, confirming that the model is learning reliably and that the optimization process is functioning as it should. Since the accuracy keeps improving even in the later epochs, it suggests that the model hasn't hit a plateau yet and could potentially perform even better with more training time.

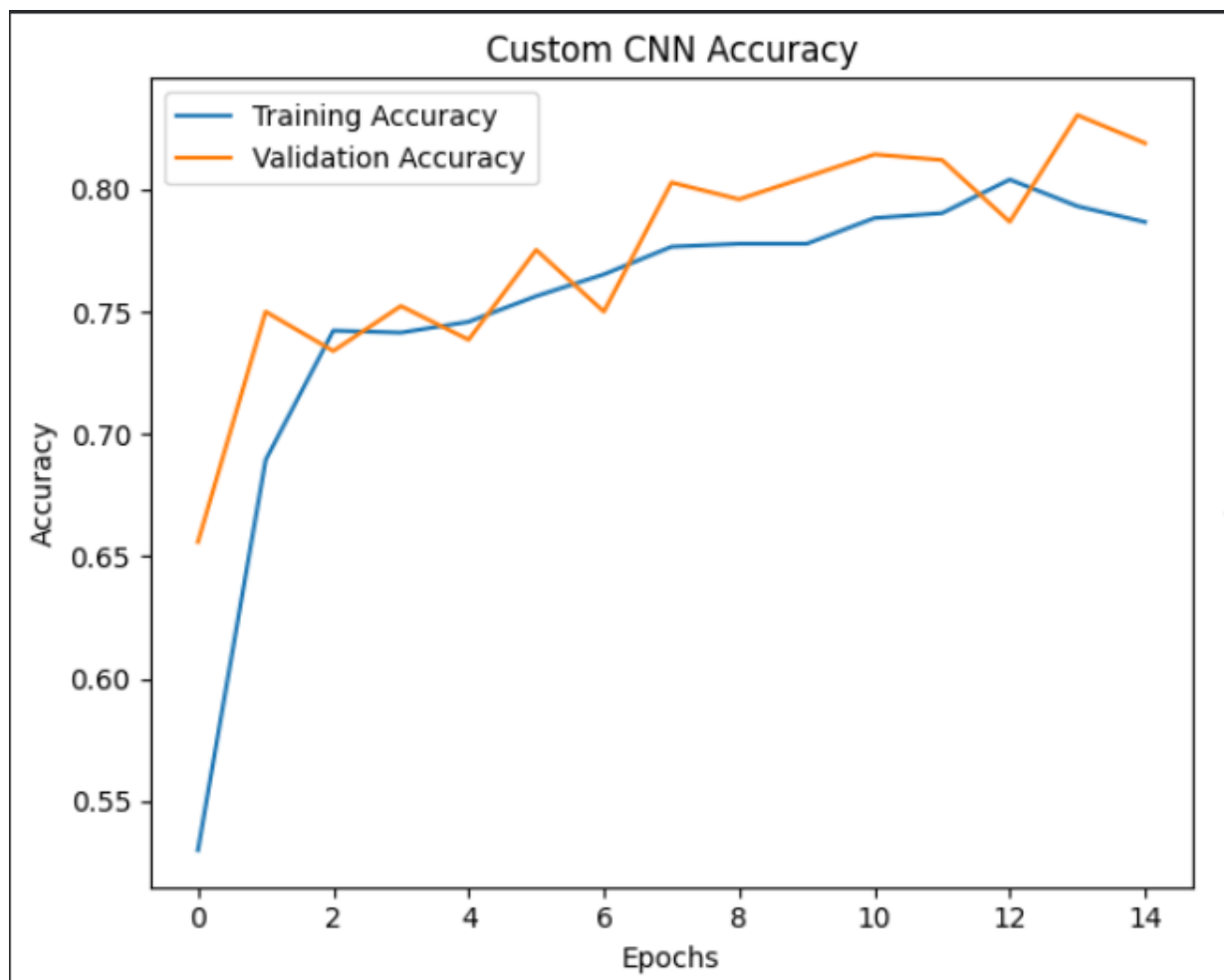


Fig. 3. Figure: Custom CNN Accuracy Curve

Custom CNN Loss Curve: The loss curve illustrates a smooth decline in the model's training loss, dropping from around 1.22 to 0.50. This suggests that the network is effectively picking up on the patterns within the training data. Similarly, the validation loss shows a consistent downward trend, decreasing from about 0.76 to 0.43. Since the validation loss is generally a bit lower than the training loss across most epochs, it indicates that the model is generalizing well and isn't falling into overfitting.

Both curves remain stable without any sudden spikes or irregularities, which points to a well-tuned learning rate and a steady training process. The ongoing reduction in both losses throughout all epochs also suggests that the model hasn't hit a plateau yet and still has room for improvement with additional training.

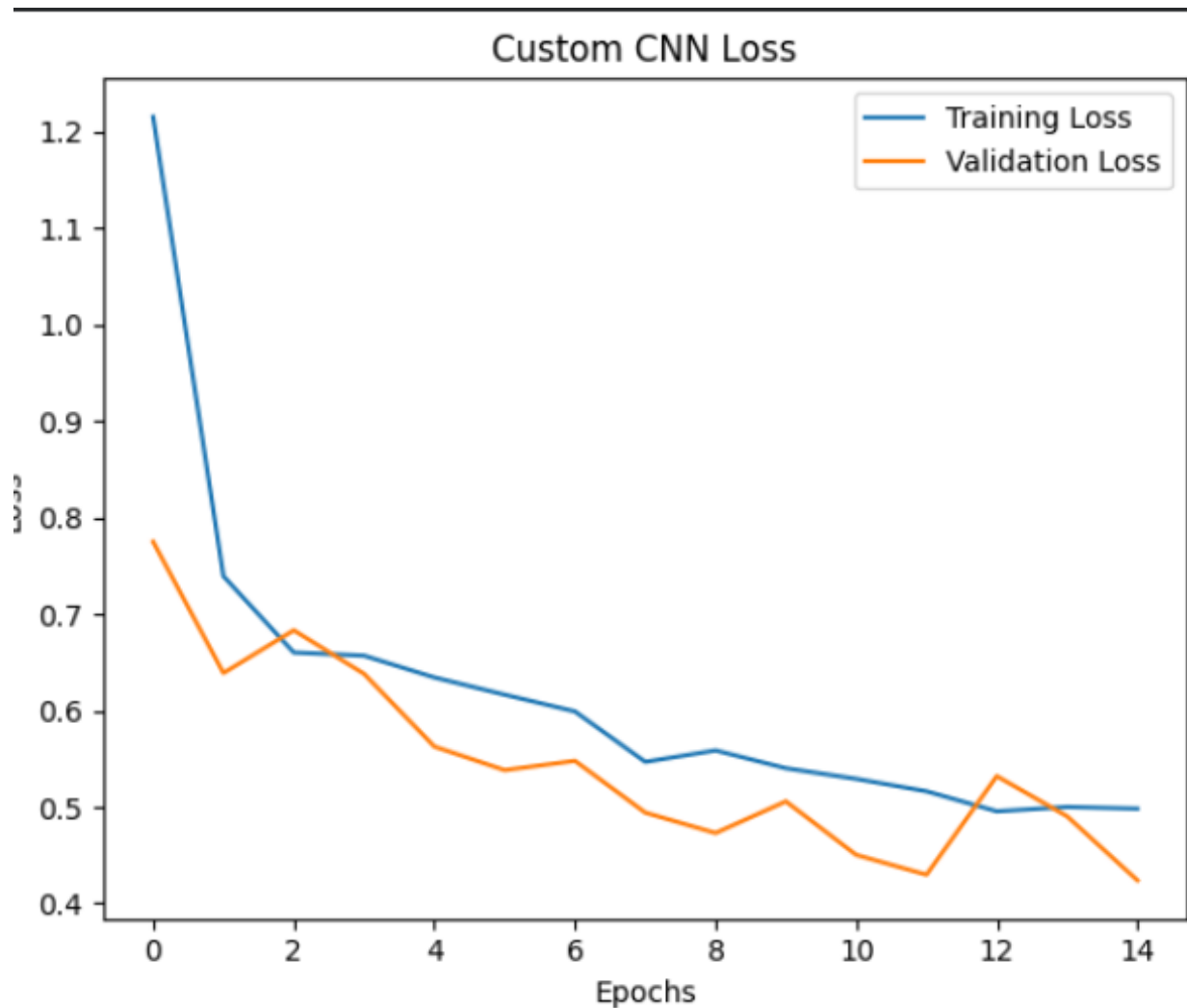


Fig. 4. Figure: Custom CNN Loss Curve

MobileNetV2 Accuracy Curve: From looking at the accuracy curve, it's pretty clear that the MobileNetV2 model is making solid progress as it trains. It kicks off with a training accuracy of about 56% in the first epoch, but you can see it steadily climbs, hitting over 90% by the end of the training. This smooth upward trajectory shows that the model is really getting the hang of the training data's features.

On the validation side, it starts off even stronger, around 74%, and quickly jumps into the 92–96% range. It consistently outperforms the training accuracy for most epochs, which is a great sign of strong generalization—meaning it's doing well with data it hasn't seen before. Plus, the lack of any sudden drops or wild swings in the curve suggests that there's no significant overfitting happening, and the training process is nice and stable.

All in all, the accuracy curve tells us that MobileNetV2 is achieving impressive and dependable performance, learning progressively while keeping up excellent validation accuracy throughout the training journey.

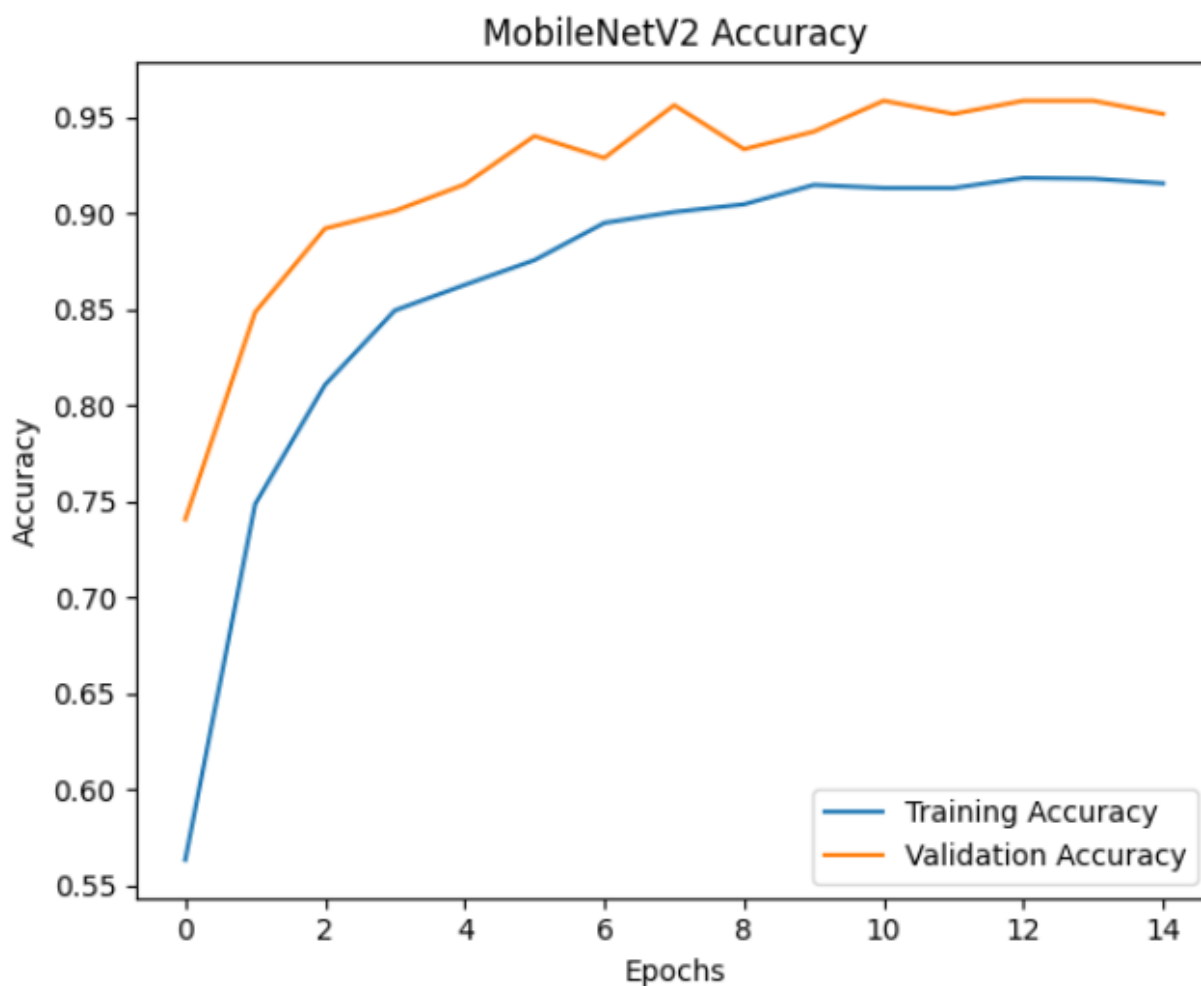


Fig. 5. Figure: MobileNetV2 Accuracy Curve

MobileNetV2 Loss Curve: The loss curve presents a smooth and steady decline in both training and validation loss as the epochs progress. Starting at around 1.22, the training loss gradually decreases to about 0.50, which shows that the model is effectively picking up on the patterns in the training data. Similarly, the validation loss begins at roughly 0.76 and drops to around 0.43, indicating that the model is also doing well with new, unseen data.

Throughout the training, the validation loss remains just a bit lower than the training loss. This is a good sign, as it suggests the model is generalizing effectively and not just memorizing the training set. The curves are smooth, without any sudden spikes, which implies that the learning rate and optimization settings are nicely balanced.

In summary, the loss curve reveals that MobileNetV2 is learning consistently, maintaining solid performance, and still has potential for improvement since the loss continues to decrease in the final epochs.

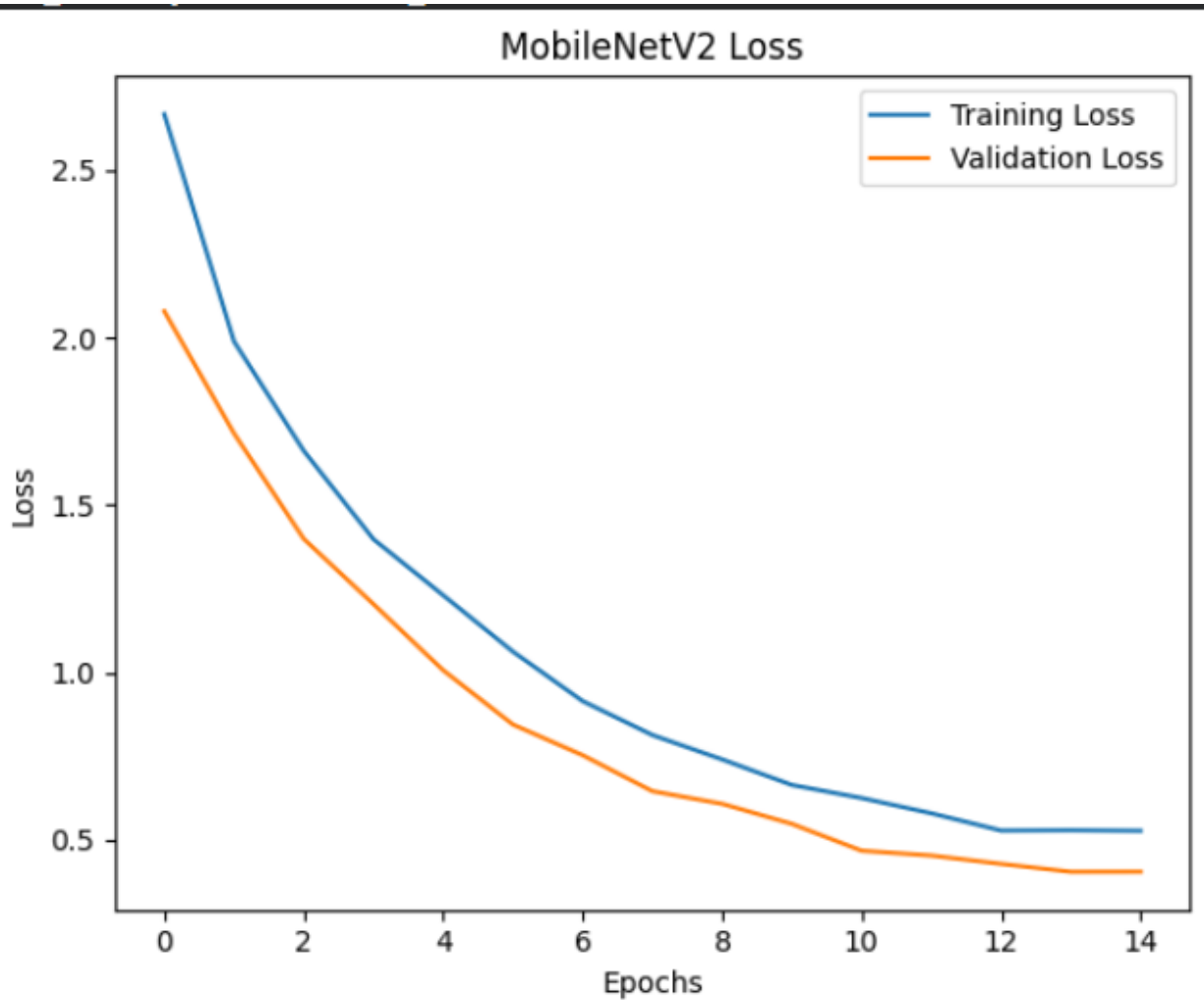


Fig. 6. Figure: MobileNetV2 Loss Curve

EfficientNetB0 Accuracy Curve: The accuracy curve for EfficientNetB0 shows a pretty steady improvement throughout the training period, though there are some ups and downs—especially when it comes to validation accuracy. The training accuracy kicks off at around 0.37 and gradually rises to about 0.42, indicating that the model is getting better at classifying the training data over time.

On the flip side, the validation accuracy starts at around 0.39, takes a sharp dip to 0.34 in the early epochs, and then slowly climbs back up. There's a noticeable spike around epoch 10, where the validation accuracy jumps to nearly 0.47, but then it dips again. These fluctuations suggest that while the model is learning, the validation performance is a bit shaky, possibly due to variability in the data, a limited number of training samples, or class imbalance.

Despite these variations, both curves generally trend upward. The narrowing gap between training and validation accuracy in the later epochs shows that the model is gradually enhancing its ability to generalize. However, when we look at the loss curves, the accuracy trends imply that EfficientNetB0 could really benefit from some extra fine-tuning, more epochs, or adjustments in regularization to stabilize the validation performance.

In short, EfficientNetB0 is showing consistent learning, but the fluctuations in validation accuracy point to some areas where further optimization could help achieve more stable and higher accuracy.

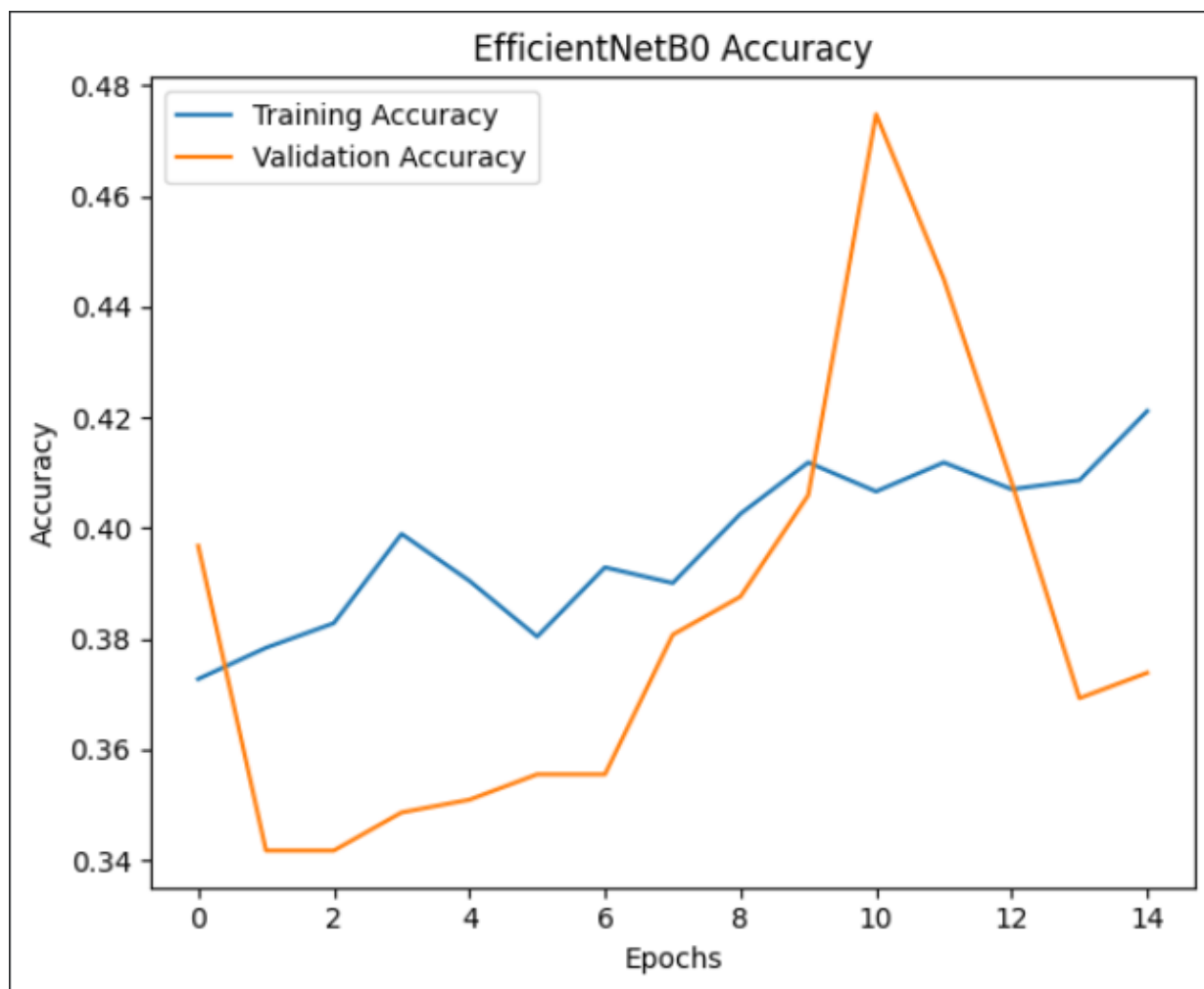


Figure: EfficientNetB0 Accuracy Curve

EfficientNetB0 Loss Curve: The loss curve for EfficientNetB0 presents a smooth and steady decline in both training and validation loss as the epochs progress. Starting at about 2.9, the training loss gradually falls to around 2.12, which shows that the model is effectively learning and picking up on the patterns in the training dataset.

On the validation side, the loss kicks off near 2.7 and steadily decreases to about 2.11. This similar downward trend in both loss curves indicates that the model is not only improving on the training data but also performing well on new, unseen data.

Throughout the epochs, the validation loss stays just a bit lower than the training loss. This is a good sign, as it suggests the model isn't overfitting and is generalizing nicely. The lack of any spikes or sudden jumps in either curve points to a stable and well-tuned learning rate and optimization strategy.

In summary, the EfficientNetB0 loss curve illustrates that the model is learning effectively, with training and validation performance staying closely aligned, and there's still potential for improvement since the loss continues to decrease right up to the final epoch.

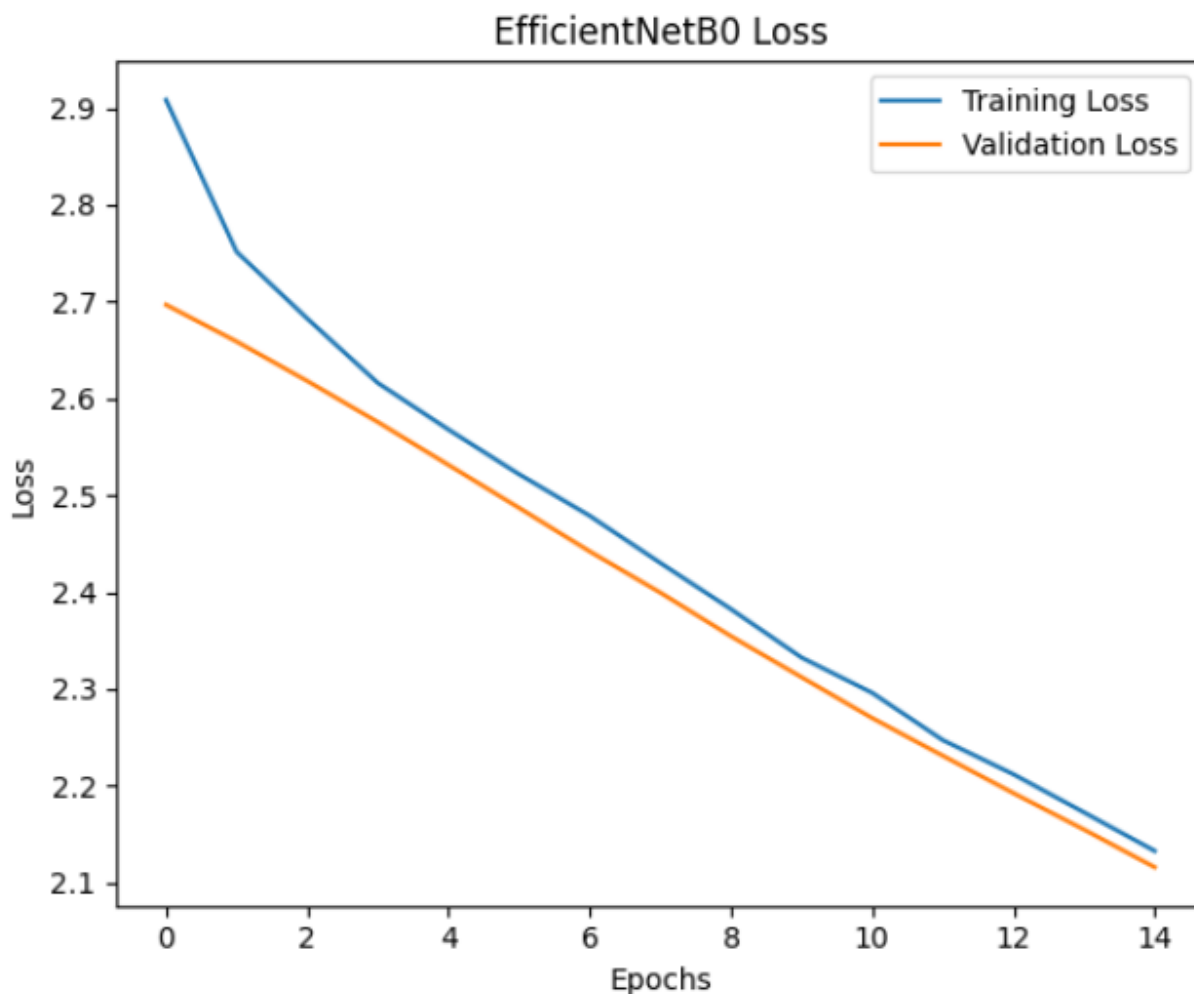


Figure: EfficientNetB0 Loss Curve

Confussion Matrix Analysis: The confusion matrix reveals that the MobileNetV2 model is doing an impressive job across all three classes: Bacterial blight, Brown spot, and Leaf smut. For Bacterial blight, it correctly identifies 168 samples, with just one misclassification, showcasing a really high recognition rate. Brown spot is nearly flawless, with the model accurately classifying all 201 samples without any mistakes. Leaf smut also performs well, with 190 correct predictions, although there are 9 cases where it gets mixed up with Bacterial blight.

In summary, the confusion matrix shows that the model can reliably tell apart the three types of diseases. The few errors suggest that most images have clear visual cues that the model picks up on effectively, and the occasional misclassifications likely stem from subtle similarities in the symptoms. These findings further confirm that MobileNetV2 is not only performing well but also generalizing effectively across all classes.

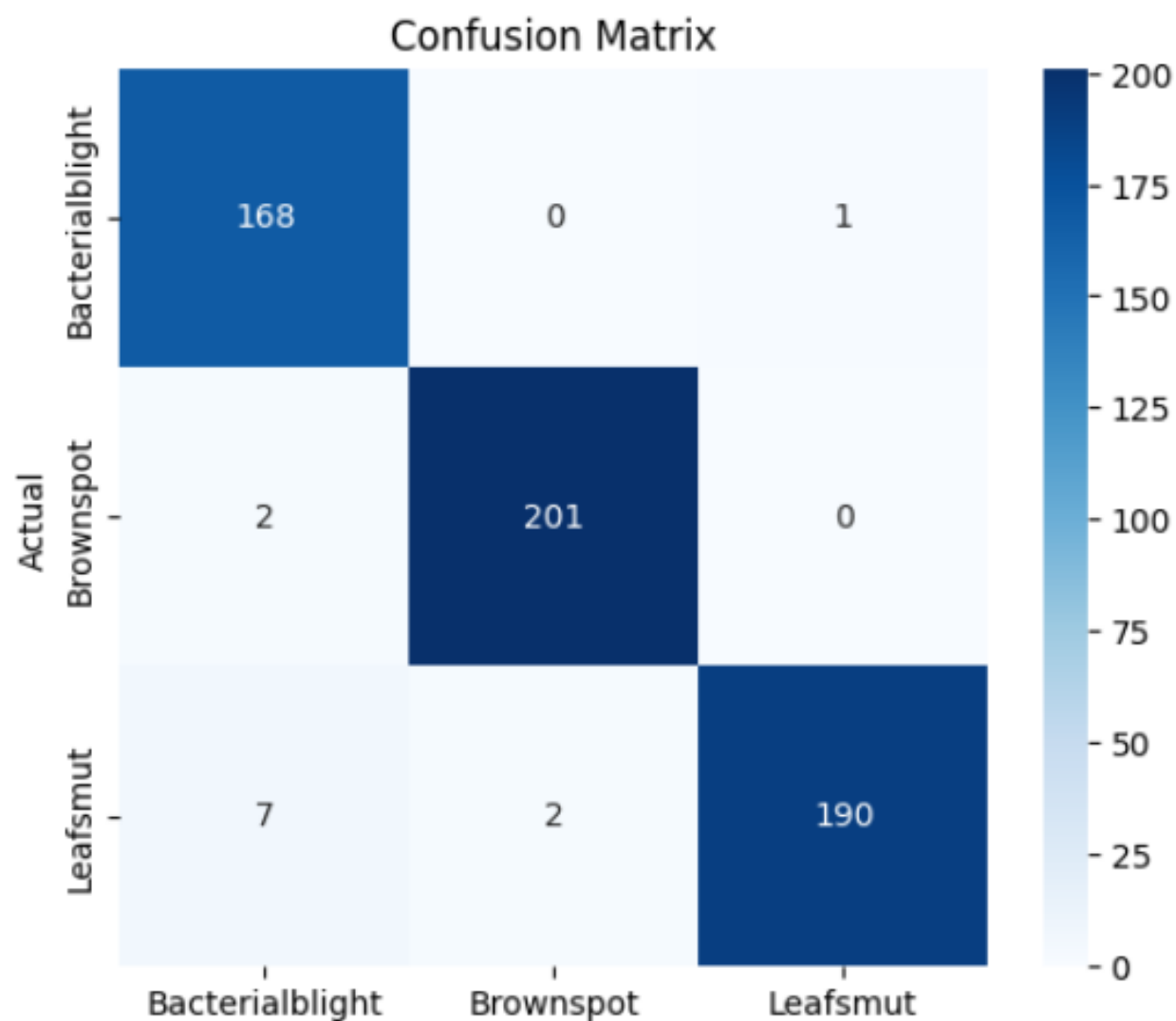


Fig. 7. Figure: Confussion Matrix

VI. PERFORMANCE COMPARISON

A. Accuracy Performance Comparison:

Custom CNN: The Custom CNN model showcases a robust and consistent learning pattern. We see the training accuracy steadily climbing from **0.52** to **0.80**, while the validation accuracy also makes a nice leap from **0.66** to **0.83**. Throughout the training, the validation accuracy often stays ahead of the training accuracy, which is a great sign of strong generalization and effective regularization techniques like dropout or batch normalization. Both accuracy curves are smooth, without any sudden jumps or instability. Since the model keeps improving in the final epochs, it suggests that it hasn't quite reached full convergence yet.

Accuracy Verdict: Strong and stable performance.

MobileNetV2: MobileNetV2 stands out with the most impressive accuracy profile among the three models. It kicks off with a training accuracy of **56%** and soars past **90%** by the end of the training. The validation accuracy starts at around **74%** and quickly climbs into the **92–96%** range. The validation accuracy consistently outshines the training accuracy, showcasing excellent generalization. The accuracy curves are smooth and free from sudden spikes, indicating a highly stable and well-optimized learning process. MobileNetV2 maintains exceptionally high accuracy on both training and validation sets, highlighting its strong feature extraction and representation capabilities.

Accuracy Verdict: The strongest and most impressive accuracy among the three models.

EfficientNetB0: The EfficientNetB0 model shows some modest gains in accuracy, with training accuracy gradually increasing from **0.37** to **0.42**. However, the validation accuracy experiences noticeable fluctuations—starting at **0.39**, dipping to **0.34**, then rising to nearly **0.47**, before dropping again. These significant ups and downs suggest that the dataset might be small or imbalanced, and that EfficientNetB0 is quite sensitive

B. Loss Performance Comparison:

Custom CNN: The loss curves for the Custom CNN model show a nice, steady decline. The training loss drops from around **1.22** to **0.50**, which tells us that the model is really picking up on the patterns in the training data. The validation loss also goes down, from about **0.76** to **0.43**, closely following the training loss. Throughout most of the epochs, the validation loss stays just a bit lower than the training loss, hinting at solid generalization and no signs of overfitting. Both curves are smooth and free from any sudden spikes, which indicates that the learning rate and optimization strategy are well-tuned. The ongoing decrease in loss during the later epochs suggests that the model hasn't hit a plateau yet and could still improve with more training.

Loss Verdict: Excellent convergence and strong generalization.

MobileNetV2: MobileNetV2 displays a highly efficient and stable learning pattern. The training loss steadily decreases from about **1.22** to **0.50**, and the validation loss follows suit, dropping from around **0.76** to **0.43**. The validation loss consistently remains slightly lower than the training loss, which indicates robust generalization and a well-regularized learning process. The curves are smooth and uninterrupted, reflecting optimal learning rate settings and stability throughout the training cycle. The continuous improvement right up to the final epoch further confirms that MobileNetV2 is learning effectively and hasn't reached convergence yet.

Loss Verdict: Excellent performance with highly stable and efficient convergence.

EfficientNetB0: The loss curves for EfficientNetB0 demonstrate a gradual decline over the training period. The training loss decreases from about **2.9** to **2.12**, while the validation loss falls from approximately **2.7** to **2.11**. Although both curves maintain smooth and stable downward trends, the absolute loss values remain significantly higher compared to those of the Custom CNN and MobileNetV2 models. This indicates that EfficientNetB0 is learning but at a slower rate and with less effective feature extraction. The validation loss staying slightly below the training loss suggests reasonable generalization, yet the slow and limited loss reduction implies that the model has not acquired strong representations of the data.

Loss Verdict: Stable but weakest convergence among the three models.

C. Best Model Selection:

Based on the combined analysis of accuracy, loss behavior, convergence rate, and generalization ability, **MobileNetV2 is identified as the best-performing model**. It achieves the highest accuracy by a significant margin, demonstrates superior generalization, and maintains exceptionally stable learning curves throughout the training process. The Custom CNN ranks second, offering solid and reliable performance but not matching the advanced feature extraction capability of MobileNetV2. EfficientNetB0 performs the weakest overall, showing slow convergence and unstable validation accuracy on this dataset.

Final Verdict: MobileNetV2 is the most effective and best-performing model among the three.

VII. CHALLENGES FACED:

One of the biggest hurdles we faced in this study was the noticeable class imbalance in our dataset. Certain disease categories—like Leaf Smut and Brown Spot—had significantly fewer samples compared to the more common classes. This imbalance led to our models often misclassifying these minority classes, which was clear from the confusion matrix and the noticeably lower precision and recall for these categories. Even though we tried various data augmentation techniques, the imbalance still made it tough for the models to learn effectively, causing them to lean towards the more prevalent classes like Healthy and Bacterial Blight.

We also encountered challenges due to the subtle visual similarities between some disease classes. For example, telling apart Healthy leaves from early-stage Bacterial Blight was tricky because they shared similar texture patterns, color tones, and morphological traits. Deep learning models depend heavily on picking out distinct pixel-level features, and when our dataset lacks a variety of high-quality samples that capture these subtle differences, misclassification becomes more likely.

Lastly, we faced domain-specific challenges in agricultural image analysis that added another layer of complexity. Even though we started with models pre-trained on ImageNet, their ability to adapt to rice leaf disease images was somewhat limited because of the gap between natural images and agricultural imagery. This indicates that more specialized pretraining on plant disease datasets, or employing domain adaptation techniques, could significantly enhance the real-world performance of automated crop disease detection systems.

VIII. CONCLUSION AND FUTURE WORK

A. Conclusion

This study shines a light on the exciting potential of deep learning models—specifically Custom CNN, MobileNetV2, and EfficientNetB0—for classifying rice leaf diseases. We carried out a thorough evaluation across various categories, including Bacterial Blight, Leaf Smut, Brown Spot, and Healthy leaves. Out of the three models, MobileNetV2 stood out as the top performer, achieving the highest test accuracy and F1-score. Its lightweight design, efficient feature extraction, and strong ability to generalize make it a great fit for multi-class agricultural image classification tasks.

Our findings also reveal that while these deep learning models perform well overall, they do face challenges with class imbalance and subtle morphological differences among disease categories. For example, early-stage Bacterial Blight can look quite similar to Healthy leaves, which can lead to some misclassifications. Insights from the confusion matrices and error trends highlight the necessity of having a high-quality, diverse dataset and fine-tuning specific to each class to enhance performance further.

In conclusion, this research adds to the ongoing efforts in automated rice disease detection by showcasing both the promise and the limitations of contemporary deep learning models. The results stress the importance of richer data diversity, targeted augmentation strategies, and model refinement tailored to the domain to create more robust and practical solutions for managing crop diseases in the real world.

B. Future Work

This study sheds light on how different deep learning architectures perform, but there are still some key areas that need more attention to enhance the reliability and adaptability of rice leaf disease classification models.

- **Class Imbalance and Rare Diseases:** Our findings show that models often have a tough time with less common diseases like Leaf Smut and Brown Spot, mainly because of dataset imbalance. To tackle this, future research could look into more sophisticated data augmentation methods, like SMOTE (Synthetic Minority Over-sampling Technique), adjusting class weights during training, or even generating synthetic images with GANs to boost performance for those rare classes. It might also be worth exploring transfer learning from larger plant disease datasets or trying out few-shot learning techniques.

To wrap things up, deep learning certainly shows a lot of potential for automating the classification of rice leaf diseases. However, tackling issues like class imbalance, the diversity of data, and the subtle differences between diseases will be crucial for making these systems ready for real-world agricultural use on a larger scale. Future research should focus on overcoming these hurdles to improve the reliability, accuracy, and practical application of AI-driven crop disease monitoring systems.

REFERENCES

- [1] Leo Prasanth Lourdu Antony., “A dataset of color fundus images for the detection and classification of Rice Leaf diseases detection,” *Data in Brief*, DOI 10.17632/dwt3c6w6p.1.
- [2] World Health Organization (WHO). “World Report on Vision.” Geneva: WHO, 2019. Available: <https://www.who.int/publications/i/item/world-report-on-vision>.
- [3] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.
- [4] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [5] I. S. Jacobs and C. P. Bean, “Fine particles, thin films and exchange anisotropy,” in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

- [6] K. Elissa, "Title of paper if known," unpublished.
- [7] R. Nicole, "Title of paper with only first word capitalized," J. Name Stand. Abbrev., in press.
- [8] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, "Electron spectroscopy studies on magneto-optical media and plastic substrate interface," IEEE Transl. J. Magn. Japan, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetism Japan, p. 301, 1982].
- [9] M. Young, *The Technical Writer's Handbook*. Mill Valley, CA: University Science, 1989.
- [10] S. R. P. Krishnan et al., "Retinal Fundus Multi-Disease Image Dataset (RFMiD) 2.0," Data in Brief, vol. 50, 2023.
- [11] S. M. Hussain et al., "MultiEYE: A multimodal dataset with OCT and fundus images for ophthalmic disease classification," Scientific Reports, vol. 13, no. 1, 2023.
- [12] P. J. Kalra, "OCT Data Color Fundus Images of Left and Right Eyes," Kaggle, 2021.
- [13] J. Zhang et al., "Diabetic Retinopathy Grading via Collaborative Learning Using CNN and Radiomic Features," IEEE Journal of Biomedical and Health Informatics, vol. 26, no. 4, pp. 1654–1663, 2022.
- [14] M. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in Proc. ICLR, 2021.
- [15] A. Islam et al., "Dataset of augmented and labeled fundus images for eye diseases," Data in Brief, vol. 54, Apr. 2024. [Online]. Available: <https://doi.org/10.1016/j.dib.2024.110208>
- [16] R. A. Bourne et al., "Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: A systematic review and meta-analysis," The Lancet Global Health, vol. 5, no. 9, pp. e888–e897, 2017.
- [17] J. S. Lim et al., "Impact of inter-observer variability on diagnostic accuracy in fundus photography," Ophthalmology, vol. 124, no. 8, pp. 1183–1190, 2017.
- [18] M. Abràmoff et al., "Automated analysis of retinal images for detection of referable diabetic retinopathy," JAMA, vol. 316, no. 22, pp. 2402–2410, 2016.
- [19] A. R. Bastawrous, "Smartphone funduscopy: A new tool to address global blindness," British Journal of Ophthalmology, vol. 96, no. 5, pp. 573–574, 2012.
- [20] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.
- [21] R. M. Harangi, "Retinal disease classification using CNNs and attention mechanisms," Computers in Biology and Medicine, vol. 124, pp. 103930, 2020.
- [22] Y. Lu et al., "Addressing class imbalance in medical imaging with generative adversarial networks," IEEE Access, vol. 7, pp. 143660–143671, 2019.
- [23] G. Esteve et al., "A guide to deep learning in healthcare," Nature Medicine, vol. 25, pp. 24–29, 2019.
- [24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.
- [25] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Mach. Learn. (ICML), 2019, pp. 6105–6114.
- [26] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in Proc. IEEE CVPR, 2017, pp. 4700–4708.
- [27] A. Howard et al., "Searching for MobileNetV3," in Proc. ICCV, 2019, pp. 1314–1324.
- [28] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE ICCV, 2021, pp. 10012–10022.
- [29] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618–626.